



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

《数据结构课程设计》

实验报告

实验 5: 排序

姓 名: XX

学 号: XX

专 业: 网络空间安全

实验时间: 2024.12.26-2025.1.2

指导老师: XX

杭州电子科技大学
网络空间安全学院

一、实验目的

- (1) 掌握简单插入排序和 Shell 排序的实现;
- (2) 掌握简单交换排序与快速排序的实现;
- (3) 掌握简单选择排序和堆排序的实现;

二、实验内容与实验结果

实验 5-1

1) 实验内容

1. 基本操作函数的实现

根据定义的记录表存储结构，编写下列基本操作函数的 C/C++源代码：

(1) 编写一个创建待排序记录表的函数 `CreateRecordTable (RecordTable &RT)`，创建的记录表保留“哨兵”，即记录从下标=1 的数组元素开始存放。

注：待排序记录数据的可以直接放在该函数中。参照实验 4-1 的静态查找表的创建方法和记录数据。

(2) 编写一个输出函数 `OutRecordTable (RecordTable RT)`，输出排序表 RT 的所有记录。

(3) 编写一个简单插入排序函数 `InsertSort (RecordTable &RT)`，将记录表 RT 中原来无序的记录按记录关键字值作正序排序，并利用全局变量 `ccount` 和 `mcoun`t 计算排序过程中关键字的比较次数和记录的移动次数（注：记录移到“哨兵”和从“哨兵”移回不算移动）。

(4) 编写一个 Shell 排序函数 `ShellSort (RecordTable &RT)`，将记录表 RT 中原来无序的记录按记录关键字值作正序排序，并利用全局变量 `ccount` 和 `mcoun`t 计算排序过程中关键字的比较次数和记录的移动次数。同时输出每一趟 Shell 排序后的记录表内容（即对每个 d 输出一次）。

2. 编写一个主函数 `main()`，检验上述操作函数是否正确，实现以下操作：

(1) 定义一个排序表 RT1，调用 `CreateRecordTable ()` 函数创建待排序记录表 RT1，将下列各记录的数据写入 RT1，然后调用 `OutRecordTable ()` 函数输出 RT1 的各记录的数据。

学号 key	姓名 name	性别 sex	年龄 age
56	Zhang	F	19
19	Wang	F	20
80	Zhou	F	19
5	Huang	M	20
21	Zheng	M	20
64	Li	M	19
88	Liu	F	18
13	Qian	F	19

37	Sun	M	20
75	Zhao	M	20
92	Chen	M	20

(2) 调用 InsertSort() 函数对 RT1 进行记录排序, 然后调用 OutRecordTable() 函数输出排序后 RT1 的各记录的数据、关键字比较次数和记录移到次数。

(3) 调用 CreateRecordTable() 函数恢复待排序记录表 RT1, 调用 ShellSort() 函数对 RT1 进行记录排序, 然后调用 OutRecordTable() 函数输出排序后 RT1 的各记录的数据、关键字比较次数和记录移到次数。

附录: 记录表创建与输出源码

```
Status CreateRecordTable(RecordTable &L) {    //创建顺序表
    int keys[]={56,19,80, 5,21,64,88,13,37,75,92};
    //int keys[]={25,12,9,20,7,31,24,35,17,10,5};
    const char
*names[]={“Zhang”,“Wang”,“ZHou”,“Huang”,“Zheng”,“Li”,“Liu”,“Qian”,“Sun”,“Zhao”,“
Chen”};
    const char *sexs[]={“F”,“F”,“F”,“F”,“M”,“M”,“M”,“M”,“M”,“M”,“M”,“M”};
    int ages[]={19,18,19,18,19,20,20,19,18,19,18};
    int i,n=11;
    for(i=1;i<=n;i++){
        L.r[i].key=keys[i-1];
        L.r[i].name=names[i-1];
        L.r[i].sex=sexs[i-1];
        L.r[i].age=ages[i-1];
    }
    L.length=n;
    return OK;
}

Status OutRecordTable(RecordTable L) {    //输出顺序表的各个记录
    int i;
    printf(“学号 姓名 性别 年龄\n”);
    for(i=1;i<=L.length;i++){
        printf(“ %2d ”,L.r[i].key);
        printf(“%5s      ”,L.r[i].name);
        printf(“%1s      ”,L.r[i].sex);
        printf(“%2d\n”,L.r[i].age);
    }
}
```

2) 源程序代码 (*.CPP)

见附件 (5-1.cpp)

3) 运行结果

原始记录表:

学号	姓名	性别	年龄
56	Zhang	F	19
19	Wang	F	18
80	Zhou	F	19
5	Huang	F	18
21	Zheng	M	19
64	Li	M	20
88	Liu	M	20
13	Qian	M	19
37	Sun	M	18
75	Zhao	M	19
92	Chen	M	18

使用插入排序结果:

学号	姓名	性别	年龄
5	Huang	F	18
13	Qian	M	19
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

比较次数: 10, 移动次数: 19

使用Shell排序结果:

Gap: 5

学号	姓名	性别	年龄
56	Zhang	F	19
19	Wang	F	18
13	Qian	M	19
5	Huang	F	18
21	Zheng	M	19
64	Li	M	20
88	Liu	M	20
80	Zhou	F	19
37	Sun	M	18
75	Zhao	M	19
92	Chen	M	18

Gap: 2

学号	姓名	性别	年龄
13	Qian	M	19
5	Huang	F	18
21	Zheng	M	19
19	Wang	F	18
37	Sun	M	18
64	Li	M	20
56	Zhang	F	19
75	Zhao	M	19
88	Liu	M	20
80	Zhou	F	19
92	Chen	M	18

```

Gap: 1
学号 姓名 性别 年龄
 5  Huang  F  18
13  Qian   M  19
19  Wang   F  18
21  Zheng  M  19
37  Sun     M  18
56  Zhang  F  19
64  Li      M  20
75  Zhao   M  19
80  Zhou   F  19
88  Liu    M  20
92  Chen   M  18
学号 姓名 性别 年龄
 5  Huang  F  18
13  Qian   M  19
19  Wang   F  18
21  Zheng  M  19
37  Sun     M  18
56  Zhang  F  19
64  Li      M  20
75  Zhao   M  19
80  Zhou   F  19
88  Liu    M  20
92  Chen   M  18
比较次数：25，移动次数：11

```

实验 5-2

1) 实验内容

1. 基本操作函数的实现

根据定义的记录表存储结构，编写下列基本操作函数的 C/C++ 源代码：

(1) 编写一个创建待排序记录表的函数 `CreateRecordTable(RecordTable &RT)`，创建的记录表保留“哨兵”，即记录从下标=1 的数组元素开始存放。

注：待排序记录数据的可以直接放在该函数中。参照实验 4-1 的静态查找表的创建方法和记录数据。

(2) 编写一个输出函数 `OutRecordTable(RecordTable RT)`，输出排序表 RT 的所有记录。

(3) 编写一个简单交换（冒泡）排序函数 `BubbleSort(RecordTable &RT)`，将记录表 RT 中原来无序的记录按记录关键字值作正序排序，并利用全局变量 `ccount` 和 `mcount` 计算排序过程中关键字的比较次数和记录的交换次数。

(4) 编写一个快速排序函数 `QuickSort(RecordTable &RT, int low, int high)`，将记录表 RT 中原来无序的记录按记录关键字值作正序排序，参数 `low` 和 `high` 表示排序记录的下标范围，并利用全局变量 `ccount` 和 `mcount` 计算排序过程中关键字的比较次数和记录的交换次数。同时输出每一次划分的序号和划分后的记录表内容（可以利用全局变量 `pcount` 计算划分的次数）。

2. 编写一个主函数 `main()`，检验上述操作函数是否正确，实现以下操作：

(1) 定义一个排序表 RT1，调用 `CreateRecordTable()` 函数创建待排序记录表 RT1，将下列各记录的数据写入 RT1，然后调用 `OutRecordTable()` 函数输出 RT1 的各记录的数据。

学号 key	姓名 name	性别 sex	年龄 age
56	Zhang	F	19
19	Wang	F	20
80	Zhou	F	19
5	Huang	M	20
21	Zheng	M	20
64	Li	M	19
88	Liu	F	18
13	Qian	F	19
37	Sun	M	20
75	Zhao	M	20
92	Chen	M	20

（2）调用 BubbleSort() 函数对 RT1 进行记录排序，然后调用 OutRecordTable() 函数输出排序后 RT1 的各记录的数据、关键字比较次数和记录移到次数。

（3）调用 CreateRecordTable() 函数恢复待排序记录表 RT1，调用 QuickSort() 函数对 RT1 进行记录排序，然后调用 OutRecordTable() 函数输出排序后 RT1 的各记录的数据、关键字比较次数和记录移到次数。

2) 源程序代码 (*.CPP)

见附件 (5-2.cpp)

3) 运行结果

原始记录表:

学号 姓名 性别 年龄

56	Zhang	F	19
19	Wang	F	18
80	Zhou	F	19
5	Huang	F	18
21	Zheng	M	19
64	Li	M	20
88	Liu	M	20
13	Qian	M	19
37	Sun	M	18
75	Zhao	M	19
92	Chen	M	18

使用冒泡排序结果:

学号 姓名 性别 年龄

5	Huang	F	18
13	Qian	M	19
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

比较次数: 55, 交换次数: 19

使用快速排序结果:

划分 #1:

学号 姓名 性别 年龄

13	Qian	M	19
19	Wang	F	18
37	Sun	M	18
5	Huang	F	18
21	Zheng	M	19
56	Zhang	F	19
88	Liu	M	20
64	Li	M	20
80	Zhou	F	19
75	Zhao	M	19
92	Chen	M	18

划分 #2:

学号 姓名 性别 年龄

5	Huang	F	18
13	Qian	M	19
37	Sun	M	18
19	Wang	F	18
21	Zheng	M	19
56	Zhang	F	19
88	Liu	M	20
64	Li	M	20
80	Zhou	F	19
75	Zhao	M	19
92	Chen	M	18

划分 #3:

学号 姓名 性别 年龄

5	Huang	F	18
13	Qian	M	19
21	Zheng	M	19
19	Wang	F	18
37	Sun	M	18
56	Zhang	F	19
88	Liu	M	20
64	Li	M	20
80	Zhou	F	19
75	Zhao	M	19
92	Chen	M	18

划分 #4:

学号 姓名 性别 年龄

5	Huang	F	18
13	Qian	M	19
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
88	Liu	M	20
64	Li	M	20
80	Zhou	F	19
75	Zhao	M	19
92	Chen	M	18

划分 #5:

学号 姓名 性别 年龄

5	Huang	F	18
13	Qian	M	19
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
75	Zhao	M	19
64	Li	M	20
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

划分 #6:

学号 姓名 性别 年龄

5	Huang	F	18
13	Qian	M	19
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

学号 姓名 性别 年龄

5	Huang	F	18
13	Qian	M	19
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

比较次数: 23, 交换次数: 9

实验 5-3

1) 实验内容

1. 基本操作函数的实现

根据定义的记录表存储结构, 编写下列基本操作函数的 C/C++源代码:

(1) 编写一个创建待排序记录表的函数 `CreateRecordTable (RecordTable &RT)`, 创建的记录表保留“哨兵”, 即记录从下标=1 的数组元素开始存放。

注：待排序记录数据的可以直接放在该函数中。参照实验 4-1 的静态查找表的创建方法和记录数据。

(2) 编写一个输出函数 `OutRecordTable(RecordTable RT)`，输出排序表 RT 的所有记录。

(3) 编写一个简单选择排序函数 `SelectSort(RecordTable &RT)`，将记录表 RT 中原来无序的记录按记录关键字值作正序排序，并利用全局变量 `ccount` 和 `mcount` 计算排序过程中关键字的比较次数和记录的交换次数。

(4) 编写一个堆排序函数 `HeapSort(RecordTable &H)`，将记录表 RT 中原来无序的记录按记录关键字值作正序排序，并利用全局变量 `ccount` 和 `mcount` 计算排序过程中关键字的比较次数和记录的交换次数。同时输出初始建立的大顶堆和每次调整堆顶元素后的记录表内容。

2. 编写一个主函数 `main()`，检验上述操作函数是否正确，实现以下操作：

(1) 定义一个排序表 RT1，调用 `CreateRecordTable()` 函数创建待排序记录表 RT1，将下列各记录的数据写入 RT1，然后调用 `OutRecordTable()` 函数输出 RT1 的各记录的数据。

学号 key	姓名 name	性别 sex	年龄 age
56	Zhang	F	19
19	Wang	F	20
80	Zhou	F	19
5	Huang	M	20
21	Zheng	M	20
64	Li	M	19
88	Liu	F	18
13	Qian	F	19
37	Sun	M	20
75	Zhao	M	20
92	Chen	M	20

(2) 调用 `SelectSort()` 函数对 RT1 进行记录排序，然后调用 `OutRecordTable()` 函数输出排序后 RT1 的各记录的数据、关键字比较次数和记录移到次数。

(3) 调用 `CreateRecordTable()` 函数恢复待排序记录表 RT1，调用 `HeapSort()` 函数对 RT1 进行记录排序，然后调用 `OutRecordTable()` 函数输出排序后 RT1 的各记录的数据、关键字比较次数和记录移到次数。

2) 源程序代码 (*.CPP)

见附件 (5-3.cpp)

3) 运行结果

原始记录表：

学号	姓名	性别	年龄
56	Zhang	F	19
19	Wang	F	18
80	Zhou	F	19
5	Huang	F	18
21	Zheng	M	19
64	Li	M	20
88	Liu	M	20
13	Qian	M	19
37	Sun	M	18
75	Zhao	M	19
92	Chen	M	18

使用简单选择排序结果：

学号	姓名	性别	年龄
5	Huang	F	18
13	Qian	M	19
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

比较次数：55，交换次数：9

初始大顶堆：

学号	姓名	性别	年龄
92	Chen	M	18
75	Zhao	M	19
88	Liu	M	20
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
80	Zhou	F	19
13	Qian	M	19
5	Huang	F	18
19	Wang	F	18
21	Zheng	M	19

调整后的堆：

学号	姓名	性别	年龄
88	Liu	M	20
75	Zhao	M	19
80	Zhou	F	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
21	Zheng	M	19
13	Qian	M	19
5	Huang	F	18
19	Wang	F	18
92	Chen	M	18

调整后的堆：

学号	姓名	性别	年龄
----	----	----	----

80	Zhou	F	19
----	------	---	----

75	Zhao	M	19
----	------	---	----

64	Li	M	20
----	----	---	----

37	Sun	M	18
----	-----	---	----

56	Zhang	F	19
----	-------	---	----

19	Wang	F	18
----	------	---	----

21	Zheng	M	19
----	-------	---	----

13	Qian	M	19
----	------	---	----

5	Huang	F	18
---	-------	---	----

88	Liu	M	20
----	-----	---	----

92	Chen	M	18
----	------	---	----

调整后的堆：

学号	姓名	性别	年龄
----	----	----	----

75	Zhao	M	19
----	------	---	----

56	Zhang	F	19
----	-------	---	----

64	Li	M	20
----	----	---	----

37	Sun	M	18
----	-----	---	----

5	Huang	F	18
---	-------	---	----

19	Wang	F	18
----	------	---	----

21	Zheng	M	19
----	-------	---	----

13	Qian	M	19
----	------	---	----

80	Zhou	F	19
----	------	---	----

88	Liu	M	20
----	-----	---	----

92	Chen	M	18
----	------	---	----

调整后的堆：

学号	姓名	性别	年龄
----	----	----	----

64	Li	M	20
----	----	---	----

56	Zhang	F	19
----	-------	---	----

21	Zheng	M	19
----	-------	---	----

37	Sun	M	18
----	-----	---	----

5	Huang	F	18
---	-------	---	----

19	Wang	F	18
----	------	---	----

13	Qian	M	19
----	------	---	----

75	Zhao	M	19
----	------	---	----

80	Zhou	F	19
----	------	---	----

88	Liu	M	20
----	-----	---	----

92	Chen	M	18
----	------	---	----

调整后的堆：

学号	姓名	性别	年龄
----	----	----	----

56	Zhang	F	19
----	-------	---	----

37	Sun	M	18
----	-----	---	----

21	Zheng	M	19
----	-------	---	----

13	Qian	M	19
----	------	---	----

5	Huang	F	18
---	-------	---	----

19	Wang	F	18
----	------	---	----

64	Li	M	20
----	----	---	----

75	Zhao	M	19
----	------	---	----

80	Zhou	F	19
----	------	---	----

88	Liu	M	20
----	-----	---	----

92	Chen	M	18
----	------	---	----

调整后的堆:

学号 姓名 性别 年龄

37	Sun	M	18
19	Wang	F	18
21	Zheng	M	19
13	Qian	M	19
5	Huang	F	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

调整后的堆:

学号 姓名 性别 年龄

21	Zheng	M	19
19	Wang	F	18
5	Huang	F	18
13	Qian	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

调整后的堆:

学号 姓名 性别 年龄

19	Wang	F	18
13	Qian	M	19
5	Huang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

调整后的堆:

学号 姓名 性别 年龄

13	Qian	M	19
5	Huang	F	18
19	Wang	F	18
21	Zheng	M	19
37	Sun	M	18
56	Zhang	F	19
64	Li	M	20
75	Zhao	M	19
80	Zhou	F	19
88	Liu	M	20
92	Chen	M	18

调整后的堆：

学号 姓名 性别 年龄

5 Huang F 18

13 Qian M 19

19 Wang F 18

21 Zheng M 19

37 Sun M 18

56 Zhang F 19

64 Li M 20

75 Zhao M 19

80 Zhou F 19

88 Liu M 20

92 Chen M 18

学号 姓名 性别 年龄

5 Huang F 18

13 Qian M 19

19 Wang F 18

21 Zheng M 19

37 Sun M 18

56 Zhang F 19

64 Li M 20

75 Zhao M 19

80 Zhou F 19

88 Liu M 20

92 Chen M 18

比较次数：31，交换次数：31

三、实验小结

在这次实验中，我通过 C/C++ 编程语言深入实践了简单插入排序、冒泡排序、选择排序这三种基础排序算法，以及它们的优化版本：Shell 排序、快速排序和堆排序。从构建有序序列的插入排序，到相邻元素比较交换的冒泡排序，再到选择最小元素进行排序的选择排序，我逐步掌握了这些基础排序算法的核心逻辑。随后，我探索了 Shell 排序如何通过间隔序列优化插入排序，快速排序如何利用分治法高效排序，以及堆排序如何借助堆数据结构特性实现稳定排序。整个实验过程中，我不仅加深了对排序算法原理的理解，还学会了如何在编程中灵活运用这些算法解决实际问题，这不仅锻炼了我的编程技能，更为我后续学习更复杂的算法奠定了坚实基础。